

Problem statement- Assignment 2

Submitted by : Tanuja Mahesh Modak

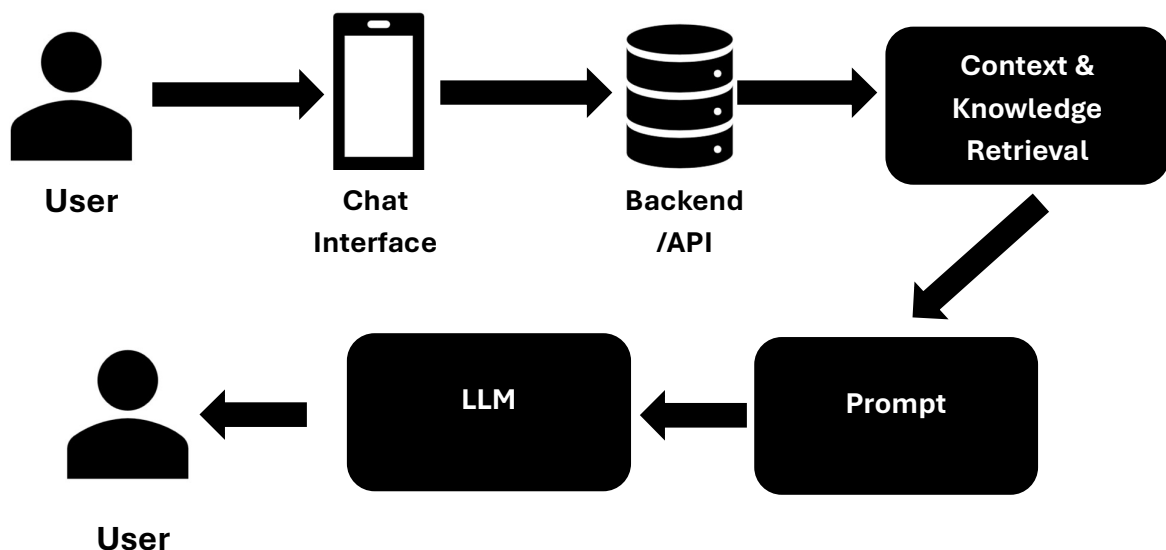
1. *Where would you rate yourself on (LLM, Deep Learning, AI, ML). A, B, C [A = can code independently; B = can code under supervision; C = have little or no understanding]*

- **LLM - B:** I understand the fundamentals of LLMs and can work with LLM-based applications under supervision.
- **Deep Learning - B:** I have a good understanding of deep-learning concepts and practical exposure to neural-network-based projects, while advanced implementations may require guidance.
- **AI - B:** I understand core AI concepts and have applied AI techniques in academic and project work, with guidance for more advanced applications.
- **ML - A:** I can independently implement the basic machine-learning workflow, including data preprocessing, feature engineering, model training, evaluation, and visualization.

2. *What are the key architectural components to create a chatbot based on LLM? Please explain the approach on a high-level*

An LLM-based chatbot consists of several components that work together to understand a user's query, retrieve relevant information when required, generate a response, and return it to the user.

A high-level architecture can be represented as:



Key Components

1. User Interface

The user interacts with the chatbot through a web or mobile interface. The interface sends the user's question to the backend and displays the generated response.

2. **Backend/API Layer**

The backend receives the request and manages the chatbot workflow. It can handle authentication, conversation management, API requests, and communication with other services.

3. **Conversation History / Memory**

Previous messages can be maintained as context so that the chatbot can understand follow-up questions and maintain continuity throughout the conversation.

4. **Knowledge Base and Retrieval**

If the chatbot needs to answer questions using specific or private information, relevant documents can be stored in a knowledge base. An embedding model can convert documents and queries into vectors, which can then be searched using a vector database. This approach is commonly known as **Retrieval-Augmented Generation (RAG)**.

5. **Prompt Construction**

The system combines the user's question, relevant conversation history, retrieved information, and system instructions to create a prompt for the LLM.

6. **Large Language Model (LLM)**

The LLM processes the prompt and generates a natural-language response. Depending on the application, the LLM could be accessed through an API or deployed locally.

7. **Tools / External APIs**

The chatbot can optionally use external tools such as calculators, databases, search systems, or business APIs when information or actions beyond the LLM are required.

8. **Safety and Response Handling**

The generated response can be checked for inappropriate content, sensitive information, or unsupported claims before being shown to the user. The system should also allow the chatbot to indicate when it does not have enough information to answer reliably.

9. **Monitoring and Evaluation**

User feedback, response quality, latency, and errors can be monitored to evaluate and continuously improve the chatbot.

High-Level Approach

I would first define the purpose and users of the chatbot and select an appropriate LLM. I would then build a backend that manages the conversation and connects the LLM with the required data sources or tools. For a chatbot that needs domain-specific or frequently changing information, I would use a RAG approach: documents would be processed into smaller sections, converted into embeddings, stored in a vector database, and retrieved when relevant to a user's question. The retrieved information and conversation context would then be included in the prompt sent to the LLM.

Finally, I would add appropriate security checks, response validation, logging, and evaluation so that the chatbot is reliable and can be improved over time.

3. Please explain vector databases. If you were to select a vector database for a hypothetical problem (you may define the problem) which one will you choose, and why?

A **vector database** is a database designed to store and search **vector embeddings**. An embedding is a numerical representation of data such as text, documents, images, or other information. Unlike traditional keyword search, vector search can identify content based on **semantic similarity**, meaning it can find information with a similar meaning even when the exact words are different.

For example, the questions “How can I apply for leave?” and “What is the procedure for taking time off?” use different words but have a similar meaning. An embedding model converts these questions into vectors, and the vector database can find documents whose vectors are closest to the query vector. Common similarity measures include cosine similarity, dot product, and Euclidean distance.

Hypothetical Problem and My Choice

Suppose I need to build an **internal company chatbot** that answers questions from product documentation, HR policies, and technical guides. I would use a **Retrieval-Augmented Generation (RAG)** approach. The documents would be divided into smaller sections, converted into embeddings, and stored in a vector database. When a user asks a question, the system retrieves the most relevant sections and provides them to the LLM as context for generating the answer.

For this problem, I would choose **Qdrant** because it is designed for vector similarity search, supports metadata filtering, can be self-hosted, and integrates well with Python-based AI applications. Metadata filtering would also be useful if documents need to be filtered by factors such as department, product, or document version.

Vector Database	Why I would consider it
Qdrant	Open-source, self-hostable, strong filtering and suitable for RAG
pgvector	Good choice when the organization already uses PostgreSQL
Pinecone	Useful when a fully managed cloud solution is preferred
FAISS	Good for experimentation and smaller prototypes

Overall, I would select **Qdrant** for this hypothetical use case because it provides the vector-search capabilities required for RAG while giving the organization control over its data and deployment. In a real project, I would also compare alternatives using factors such as retrieval accuracy, latency, scalability, security, and operational requirements before making the final decision.